

Python: Standard Library Cheat Sheet

Fundamentals of the standard library — the "batteries included" modules that ship with Python itself, no installs needed. Each section shows the everyday core of one module or task; the full APIs live at docs.python.org. Language topics stay on the basics and advanced sheets.

File & Console I/O

The basics sheet introduces `open()`; this is the fuller picture — appending, binary mode, and the standard streams.

```
import sys

# Console: print() writes to STDOUT; send errors to STDERR
print("normal output")          # -> stdout
print("something went wrong", file=sys.stderr)  # -> stderr
sys.stdout.write("raw write\n")  # print() without separators/newline logic
# input() reads a line from stdin (see the getting-started sheet)

# The standard streams ARE ordinary file objects — the same type open()
# returns, pre-opened by the interpreter. The whole file interface works,
# and they fit anywhere a file object is expected (e.g. print(file=...)).
type(sys.stdout)                # <class '_io.TextIOWrapper'> (when a terminal)
sys.stderr.write("oops\n")      # 5 -> write() returns the chars written
sys.stdout.writable()           # True   (stdin: read-only; stdout/err: write-only)
sys.stdin.readable()            # True
# text = sys.stdin.read()       # read stdin until EOF — would sit waiting here
# for line in sys.stdin:        # ...or iterate it line by line, like any file
# Never close() them — the interpreter owns their lifetime.

# Text files: pass an explicit encoding, and let `with` close the file
with open("notes.txt", "w", encoding="utf-8") as f:  # w = write (TRUNCATES)
    f.write("first line\n")

with open("notes.txt", "a", encoding="utf-8") as f:  # a = append to the end
    f.write("second line\n")

with open("notes.txt", encoding="utf-8") as f:      # mode "r" is the default
    content = f.read()
content      # 'first line\nsecond line\n'

# Read line by line (memory-friendly: never loads the whole file)
with open("notes.txt", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip())    # first line / second line

# Binary files: add "b" to the mode -> bytes in, bytes out, no encoding
# (the bytes type itself is covered on the basics sheet)
with open("blob.bin", "wb") as f:
    f.write(b"\x00\x01\xff")

with open("blob.bin", "rb") as f:
    data = f.read()
data      # b'\x00\x01\xff'
len(data) # 3

# Modes cheat: r read | w write (truncate!) | a append | x create, fail
```

```
# if it exists | +b modifiers combine: "rb", "a+", "xb"...
```

Command-Line Arguments

```
import sys

# sys.argv holds the arguments as a list of STRINGS; [0] is the script.
# Running: python myscript.py input.txt --fast
# gives:    sys.argv == ['myscript.py', 'input.txt', '--fast']

# argparse: declare the arguments, get parsing, validation, and --help
import argparse

parser = argparse.ArgumentParser(description="Process a file")
parser.add_argument("path") # required positional
parser.add_argument("--fast", action="store_true") # boolean flag
parser.add_argument("--level", type=int, default=1) # typed, with default

# In a real script: args = parser.parse_args() (reads sys.argv)
# An explicit list also works - used here to keep the example runnable:
args = parser.parse_args(["input.txt", "--fast", "--level", "3"])
args.path # 'input.txt'
args.fast # True
args.level # 3 -> already an int, thanks to type=int

# Bad input never reaches your code: argparse prints usage and exits
# parser.parse_args([]) # error: the following arguments are
# # required: path -> SystemExit(2)
# parser.parse_args(["-h"]) # prints full help and exits
```

Environment Variables

```
import os

# os.environ is a dict-like view of the process environment
# os.environ["HOME"] # e.g. '/Users/alice' - varies per machine
os.environ.get("API_KEY") # None -> missing key, no error (like dict.get)
os.environ.get("API_KEY", "") # "" -> or supply your own default
# os.environ["API_KEY"] # KeyError if not set ([] access raises)

# Setting: values must be STRINGS; child processes inherit them
os.environ["APP_MODE"] = "debug"
os.environ["APP_MODE"] # 'debug'
# os.environ["PORT"] = 8000 # TypeError: str expected, not int
os.environ["PORT"] = "8000" # numbers go in (and come out) as strings
int(os.environ["PORT"]) # 8000 -> convert on the way out

del os.environ["APP_MODE"] # unset
"APP_MODE" in os.environ # False

# Changes affect THIS process and its children only - never the shell
# that launched it; everything reverts when the process exits.
```

JSON

```
import json

# Python -> JSON string: dumps ("dump to string"). The typical input is
# a DICT (any mix of the mapped types below works):
data = {"name": "Alice", "age": 30, "tags": ["admin"], "active": True}
json.dumps(data)    # '{"name": "Alice", "age": 30, "tags": ["admin"], "active": true}'

# JSON string -> Python: loads. A JSON object comes back as a real DICT
# (a top-level JSON array would come back as a list):
json.loads('{"x": 1, "ok": true, "note": null}')
# {'x': 1, 'ok': True, 'note': None}    -> a dict; true/null became True/None

# Any mapped type works at the TOP level too - e.g. a list
json.dumps([1, "two", True])    # '[1, "two", true]'
json.loads("[1, 2, 3]")        # [1, 2, 3]

# Mapping: dict<->object, list/tuple<->array, str<->string, int/float
# <->number, True/False<->true/false, None<->null. Sets don't fit:
# json.dumps({1, 2})    # TypeError: Object of type set is not JSON serializable

# Class instances don't serialize by themselves either...
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

u = User("Alice", 30)
# json.dumps(u)          # TypeError: Object of type User is not
#                        #                        JSON serializable
# ...but their attribute dict does - vars(u) is u.__dict__ (see the
# advanced sheet's Introspection section):
json.dumps(vars(u))        # '{"name": "Alice", "age": 30}'
json.dumps(u, default=vars) # same result - and default= also covers
#                        # objects NESTED anywhere in the data

# Pretty-print with indent
print(json.dumps({"a": 1, "b": [2, 3]}, indent=2))
# {
#   "a": 1,
#   "b": [
#     2,
#     3
#   ]
# }

# Files: dump/load (no s) work on file objects directly
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)

with open("config.json", encoding="utf-8") as f:
    loaded = json.load(f)
loaded == data    # True -> clean round-trip

# Non-ASCII is escaped by default; ensure_ascii=False keeps it readable
json.dumps({"city": "città"})    # '{"city": "citt\\u00e0"}'
json.dumps({"city": "città"}, ensure_ascii=False) # '{"city": "città"}'

# GOTCHA: JSON object keys are ALWAYS strings - non-string keys convert
```

```
json.loads(json.dumps({1: "one"}))    # {'1': 'one'} -> the int key became '1'
```

Base64

Base64 represents arbitrary **bytes** as plain ASCII text — the standard way to move binary data through text-only channels (JSON, URLs, email). It is an encoding, **not encryption**: anyone can decode it, so it protects nothing.

```
import base64

# bytes in -> bytes out (the bytes type is on the basics sheet)
raw = b"hi there"
encoded = base64.b64encode(raw)    # b'aGkgdGhlcmU='
base64.b64decode(encoded)         # b'hi there' -> clean round-trip

# Strings need the str<->bytes hop on BOTH ends
b64 = base64.b64encode("città".encode("utf-8")).decode("ascii")
b64                                # 'Y2l0dMOg' -> a plain str now
base64.b64decode(b64).decode("utf-8")    # 'città'

# URL-safe variant: substitutes _ for +/ (safe in URLs and filenames)
base64.b64encode(bytes([251, 255]))      # b'+/8='
base64.urlsafe_b64encode(bytes([251, 255])) # b'-_8='
```